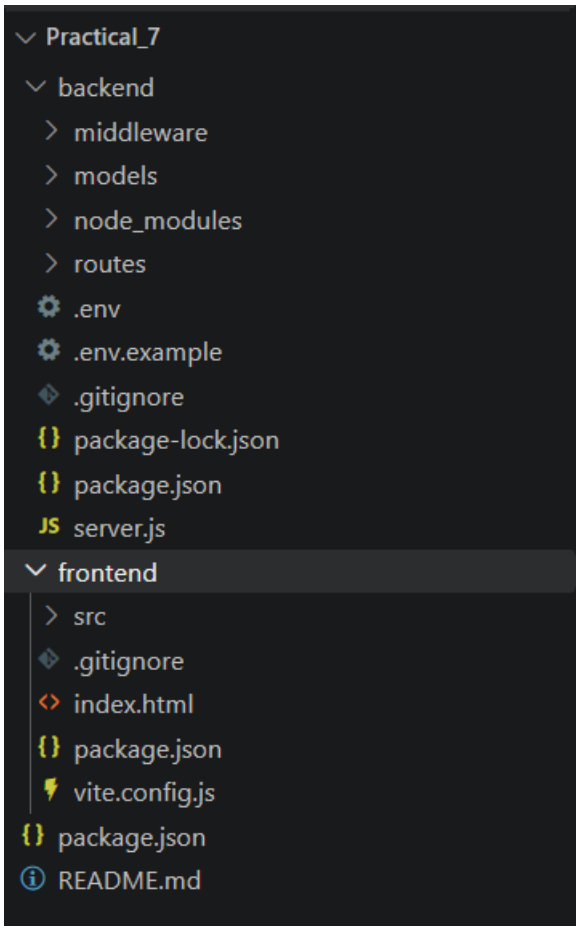
Practical 7

Problem Definition

- Add user registration and login functionality to the Task Management application.
- Hash passwords using bcrypt before saving the User document to MongoDB.
- Implement JWT token generation on successful login, with a reasonable expiry (1 hour).
- Protect all task routes using an authentication middleware that verifies the JWT.
- Add server-side input validation that rejects malformed requests (e.g., missing title, invalid email, short password) before they reach the database.

Folder Structure:



backend/models/User.js

```
const mongoose = require('mongoose');

const userSchema = new mongoose.Schema(
 {
  name: {
   type: String,
   required: [true, 'Name is required'],
   trim: true
  },
  email: {
   type: String,
   required: [true, 'Email is required'],
   unique: true,
   lowercase: true,
   trim: true,
   match: [
    /^\w+([.-]?\w+)*@\w+([.-]?\w+)*(\.\w{2,3})+$/,
    'Please enter a valid email address'
   ]
  },
  password: {
   type: String,
   required: [true, 'Password is required'],
   minlength: [6, 'Password must be at least 6 characters long']
  },
  createdAt: {
   type: Date,
   default: Date.now
```

```
    }
  },
  {
    timestamps: true
  }
);

// Method to sanitize output and exclude password from JSON serialization
userSchema.methods.toJSON = function () {
  const user = this.toObject();
  delete user.password;
  return user;
};

module.exports = mongoose.model('User', userSchema);
```

backend/middleware/auth.js

```
const jwt = require('jsonwebtoken');

/**
 * Authentication Middleware
 * Validates the JWT provided in the Authorization header: Bearer <token>
 * Attaches the decoded payload to req.user
 */
const authMiddleware = (req, res, next) => {
  const authHeader = req.headers.authorization;

  if (!authHeader || !authHeader.startsWith('Bearer ')) {
```

```
    return res.status(401).json({
      error: 'Access denied. No token provided or malformed header. Expected: Bearer <token>'
    });
  }

  const token = authHeader.split(' ')[1];

  if (!token) {
    return res.status(401).json({
      error: 'Access denied. Token missing.'
    });
  }

  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    req.user = decoded; // Contains { id: user._id, email: user.email, iat, exp }
    next();
  } catch (err) {
    if (err.name === 'TokenExpiredError') {
      return res.status(401).json({
        error: 'Token has expired. Please log in again.'
      });
    }
    return res.status(401).json({
      error: 'Invalid token. Authorization failed.'
    });
  }
```

```
};

module.exports = authMiddleware;
```

backend/middleware/validate.js

```
/**
 * Validation Middleware Pipeline
 * Enforces server-side validation rules before requests reach the controllers.
 */

const emailRegex = /^\w+([.-]?\w+)*@\w+([.-]?\w+)*(\.\w{2,3})+$/;

// Validate User Registration payload
const validateRegister = (req, res, next) => {
  const { name, email, password } = req.body;
  const errors = {};

  if (!name || typeof name !== 'string' || !name.trim()) {
    errors.name = 'Full name is required';
  }

  if (!email || typeof email !== 'string' || !email.trim()) {
    errors.email = 'Email address is required';
  } else if (!emailRegex.test(email.trim())) {
    errors.email = 'Please provide a valid email address';
  }
```

```
  if (!password || typeof password !== 'string') {
    errors.password = 'Password is required';
  } else if (password.length < 6) {
    errors.password = 'Password must be at least 6 characters long';
  }

  if (Object.keys(errors).length > 0) {
    return res.status(400).json({
      error: 'Validation failed',
      errors
    });
  }

  next();
};

// Validate User Login payload
const validateLogin = (req, res, next) => {
  const { email, password } = req.body;
  const errors = {};

  if (!email || typeof email !== 'string' || !email.trim()) {
    errors.email = 'Email is required';
  } else if (!emailRegex.test(email.trim())) {
    errors.email = 'Please provide a valid email address';
  }

  if (!password || typeof password !== 'string' || !password.trim()) {
```

```
    errors.password = 'Password is required';
  }

  if (Object.keys(errors).length > 0) {
    return res.status(400).json({
      error: 'Validation failed',
      errors
    });
  }

  next();
};

// Validate Task Creation and Update payload
const validateTask = (req, res, next) => {
  const { title, priority } = req.body;
  const errors = {};

  // For POST or when title is provided in PUT
  if (req.method === 'POST' || (req.method === 'PUT' && title !== undefined)) {
    if (!title || typeof title !== 'string' || !title.trim()) {
      errors.title = 'Task title is required and cannot be empty';
    }
  }

  if (priority !== undefined) {
    const validPriorities = ['low', 'medium', 'high'];
    if (!validPriorities.includes(priority)) {
```

```
    errors.priority = 'Priority must be one of: low, medium, high';
   }
  }

  if (Object.keys(errors).length > 0) {
   return res.status(400).json({
    error: 'Validation failed',
    errors
   });
  }

  next();
};

module.exports = {
  validateRegister,
  validateLogin,
  validateTask
};
```

backend/routes/auth.js

```
const express = require('express');
const bcrypt = require('bcryptjs');
const jwt = require('jsonwebtoken');
const User = require('../models/User');
const authMiddleware = require('../middleware/auth');
const { validateRegister, validateLogin } = require('../middleware/validate');
```

```
const router = express.Router();

/**
 * Helper to generate JWT token with 1 hour expiration
 */
const generateToken = (user) => {
  return jwt.sign(
    {
      id: user._id,
      email: user.email,
      name: user.name
    },
    process.env.JWT_SECRET,
    {
      expiresIn: process.env.JWT_EXPIRES_IN || '1h'
    }
  );
};

/**
 * @route   POST /auth/register
 * @desc    Register a new user, hash password with bcrypt, and return token
 * @access  Public
 */
router.post('/register', validateRegister, async (req, res, next) => {
  try {
    const { name, email, password } = req.body;
```

```
  // Check if user already exists
  const existingUser = await User.findOne({ email: email.toLowerCase() });
  if (existingUser) {
    return res.status(400).json({
      error: 'Email already registered. Please login or use a different email.'
    });
  }

  // Step 3 in lab manual: Hash password using bcrypt
  const hashedPassword = await bcrypt.hash(password, 10);

  const user = await User.create({
    name: name.trim(),
    email: email.toLowerCase().trim(),
    password: hashedPassword
  });

  const token = generateToken(user);

  res.status(201).json({
    message: 'User registered successfully',
    token,
    user: {
      id: user._id,
      name: user.name,
      email: user.email,
      createdAt: user.createdAt
    }
```

```
    });
  } catch (err) {
    next(err);
  }
});

/**
 * @route   POST /auth/login
 * @desc    Authenticate user, verify password with bcrypt, return JWT token
 * @access  Public
 */
router.post('/login', validateLogin, async (req, res, next) => {
  try {
    const { email, password } = req.body;

    // Check if user exists
    const user = await User.findOne({ email: email.toLowerCase() });
    if (!user) {
      return res.status(401).json({
        error: 'Invalid credentials. User with this email does not exist.'
      });
    }

    // Step 4 in lab manual: Compare password with bcrypt.compare
    const isMatch = await bcrypt.compare(password, user.password);
    if (!isMatch) {
      return res.status(401).json({
        error: 'Invalid credentials. Incorrect password.'
```

```
    });
  }

  const token = generateToken(user);

  res.status(200).json({
    message: 'Login successful',
    token,
    user: {
      id: user._id,
      name: user.name,
      email: user.email,
      createdAt: user.createdAt
    }
  });
 } catch (err) {
  next(err);
 }
});

/**
 * @route   GET /auth/me
 * @desc    Supplementary Problem 1: Return currently logged-in user profile
 * @access  Private (Protected with JWT Auth Middleware)
 */
router.get('/me', authMiddleware, async (req, res, next) => {
 try {
  const user = await User.findById(req.user.id).select('-password');
```

```
    if (!user) {
      return res.status(404).json({ error: 'User not found' });
    }
    res.status(200).json({
      user: {
        id: user._id,
        name: user.name,
        email: user.email,
        createdAt: user.createdAt
      }
    });
  } catch (err) {
    next(err);
  }
});

module.exports = router;
```

backend/routes/tasks.js

```
const express = require('express');
const Task = require('../models/Task');
const authMiddleware = require('../middleware/auth');
const { validateTask } = require('../middleware/validate');

const router = express.Router();

// Step 6 in lab manual: Apply the auth middleware to all task routes
router.use(authMiddleware);
```

```
/**
 * Helper: Format Mongoose validation errors
 */
function formatValidationError(err) {
  const errors = {};
  Object.keys(err.errors).forEach((field) => {
    errors[field] = err.errors[field].message;
  });
  return errors;
}

/**
 * @route   GET /tasks
 * @desc    Get all tasks belonging to the authenticated user
 * @access  Private (JWT Protected)
 */
router.get('/', async (req, res, next) => {
  try {
    const tasks = await Task.find({ user: req.user.id }).sort({ createdAt: -1 });
    res.status(200).json(tasks);
  } catch (err) {
    next(err);
  }
});

/**
 * @route   GET /tasks/:id
```

```
 * @desc    Get single task by ID for authenticated user
 * @access  Private (JWT Protected)
 */
router.get('/:id', async (req, res, next) => {
  try {
    const task = await Task.findOne({ _id: req.params.id, user: req.user.id });
    if (!task) {
      return res.status(404).json({ error: 'Task not found' });
    }
    res.status(200).json(task);
  } catch (err) {
    if (err.name === 'CastError') {
      return res.status(400).json({ error: 'Invalid task ID format' });
    }
    next(err);
  }
});

/**
 * @route   POST /tasks
 * @desc    Create a new task for the authenticated user
 * @access  Private (JWT Protected)
 */
router.post('/', validateTask, async (req, res, next) => {
  try {
    const { title, description, completed, priority } = req.body;
    const task = await Task.create({
      user: req.user.id,
```

```
      title: title.trim(),
      description: description ? description.trim() : '',
      completed: completed === true,
      priority: priority || 'medium'
    });
    res.status(201).json(task);
  } catch (err) {
    if (err.name === 'ValidationError') {
      return res.status(400).json({ errors: formatValidationError(err) });
    }
    next(err);
  }
});

/**
 * @route   PUT /tasks/:id
 * @desc    Update an existing task
 * @access  Private (JWT Protected)
 */
router.put('/:id', validateTask, async (req, res, next) => {
  try {
    const { title, description, completed, priority } = req.body;

    const task = await Task.findOneAndUpdate(
      { _id: req.params.id, user: req.user.id },
      {
        ...(title !== undefined && { title: title.trim() }),
        ...(description !== undefined && { description: description.trim() }),
```

```
      ...(completed !== undefined && { completed: Boolean(completed) }),
      ...(priority !== undefined && { priority })
    },
    {
      new: true,
      runValidators: true
    }
  );

  if (!task) {
    return res.status(404).json({ error: 'Task not found or unauthorized' });
  }

  res.status(200).json(task);
 } catch (err) {
  if (err.name === 'ValidationError') {
    return res.status(400).json({ errors: formatValidationError(err) });
  }
  if (err.name === 'CastError') {
    return res.status(400).json({ error: 'Invalid task ID format' });
  }
  next(err);
 }
});

/**
 * @route   DELETE /tasks/:id
 * @desc    Delete a task
```

```
 * @access  Private (JWT Protected)
 */
router.delete('/:id', async (req, res, next) => {
  try {
    const task = await Task.findOneAndDelete({ _id: req.params.id, user: req.user.id });
    if (!task) {
      return res.status(404).json({ error: 'Task not found or unauthorized' });
    }
    res.status(200).json({
      message: 'Task deleted successfully',
      task
    });
  } catch (err) {
    if (err.name === 'CastError') {
      return res.status(400).json({ error: 'Invalid task ID format' });
    }
    next(err);
  }
});

module.exports = router;
```

backend/server.js

```
/**
 * Practical 7: Authentication and Middleware Pipeline
 * Course: ITUE301 - Advanced Web Development Frameworks
 * Student Roll No: D25DIT090
 *
```

```
 * Objective: To implement JWT-based authentication and input validation
 * as part of the Express middleware pipeline.
 */

const express = require('express');
const mongoose = require('mongoose');
const cors = require('cors');
require('dotenv').config();

const authRoutes = require('./routes/auth');
const taskRoutes = require('./routes/tasks');

const app = express();
const PORT = process.env.PORT || 5000;

/* ─────────────────────────────────────────────────
 Middleware Pipeline
──────────────────────────────────────────────────── */

// 1. CORS Configuration (Allows React frontend at localhost:5173 to access API)
app.use(cors());

// 2. Parse JSON request bodies
app.use(express.json());

// 3. Request Logger Middleware
app.use((req, res, next) => {
  console.log(`[${new Date().toISOString()}] ${req.method} ${req.url}`);
```

```
  next();
});

// 4. Content-Type validation for POST & PUT write requests
app.use((req, res, next) => {
  if (
    (req.method === 'POST' || req.method === 'PUT') &&
    req.headers['content-type'] !== 'application/json'
  ) {
    return res.status(400).json({
      error: 'Content-Type must be application/json for POST and PUT requests'
    });
  }
  next();
});

/* ————————————————————————————————————
  Mount Routes
———————————————————————————————————— */
// Public / Auth routes (register, login, me)
app.use('/auth', authRoutes);

// Protected Task CRUD routes (guarded by authMiddleware inside taskRoutes)
app.use('/tasks', taskRoutes);

// Health check endpoint
app.get('/health', (req, res) => {
  res.status(200).json({
```

```
    status: 'online',
    timestamp: new Date().toISOString(),
    service: 'ITUE301 Practical 7 Auth & Task API'
  });
});

/* ————————————————————————————————————
 404 Handler – Undefined routes
———————————————————————————————————— */
app.use((req, res) => {
  res.status(404).json({
    error: `Route ${req.method} ${req.url} not found`
  });
});

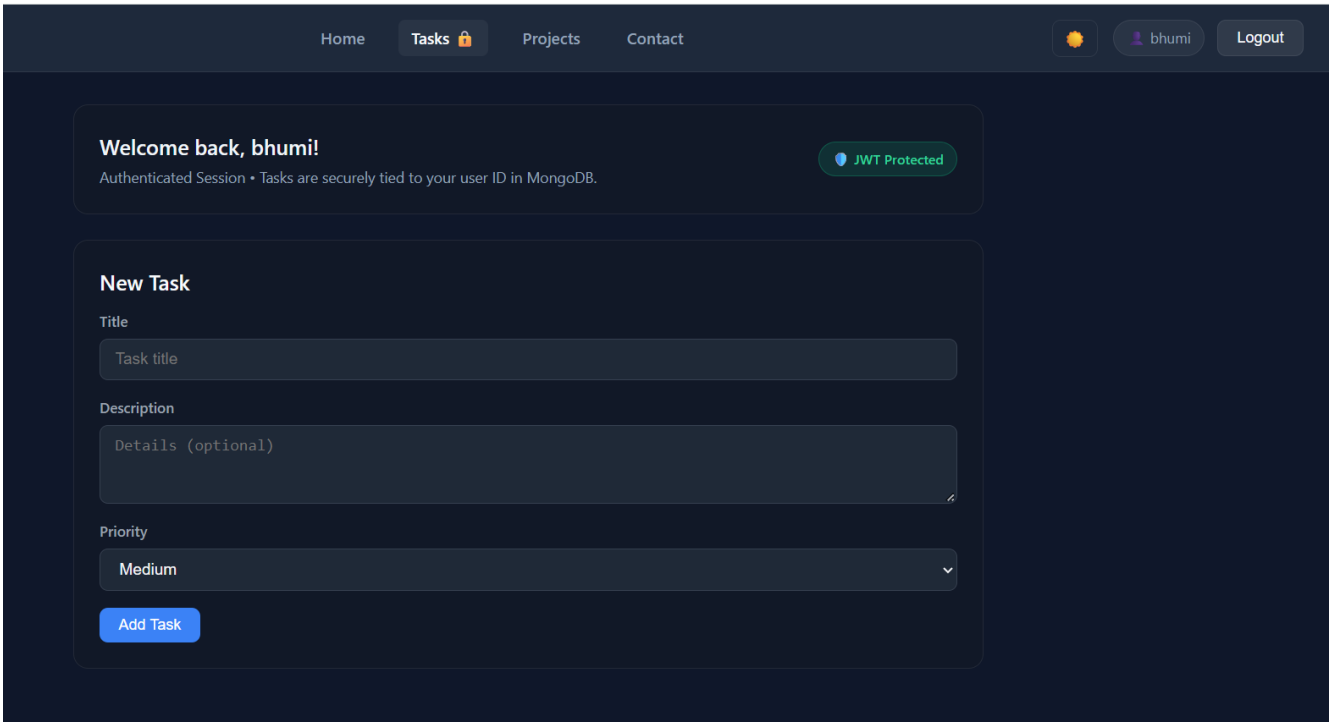
/* ————————————————————————————————————
 Global Error Handler (Must be the last middleware)
———————————————————————————————————— */
app.use((err, req, res, next) => {
  console.error(`[ERROR] ${err.stack || err.message}`);
  res.status(err.status || 500).json({
    error: err.message || 'Something went wrong on the server'
  });
});

/* ————————————————————————————————————
 Connect to MongoDB, then start the server
———————————————————————————————————— */
```

```
const MONGO_URI = process.env.MONGO_URI ||
'mongodb://localhost:27017/taskmanager_practical7';


mongoose
  .connect(MONGO_URI)
  .then(() => {
    console.log('MongoDB connected successfully');
    app.listen(PORT, () => {
      console.log(`Server running on http://localhost:${PORT}`);
      console.log('Available endpoints:');
      console.log('  POST /auth/register    - Register user (bcrypt hashed)');
      console.log('  POST /auth/login       - Login & receive JWT token');
      console.log('  GET  /auth/me          - [Protected] Get current user profile');
      console.log('  GET  /tasks            - [Protected] Get all user tasks');
      console.log('  POST /tasks            - [Protected] Create new task');
      console.log('  GET  /tasks/:id        - [Protected] Get task by ID');
      console.log('  PUT  /tasks/:id        - [Protected] Update task');
      console.log('  DELETE /tasks/:id      - [Protected] Delete task');
    });
  })
  .catch((err) => {
    console.error('MongoDB connection failed:', err.message);
    console.error('Make sure your MONGO_URI in .env is correct.');
    // Start server even if MongoDB is offline so tests and routes can be verified
    app.listen(PORT, () => {
      console.log(`Server running in offline-DB mode on http://localhost:${PORT}`);
    });
  });
```

```
module.exports = app;
```



frontend/src/api.js

```
const BASE_URL = "http://localhost:5000";

/**
 * Token Helpers
 */
export const getToken = () => localStorage.getItem("auth_token");
export const setToken = (token) => localStorage.setItem("auth_token", token);
export const removeToken = () => {
  localStorage.removeItem("auth_token");
  localStorage.removeItem("auth_user");
};
```

```
/**
 * Centralized response handler with 401 token expiry handling
 */
async function handleResponse(res) {
  if (!res.ok) {
    let message = `Request failed with status ${res.status}`;
    let errorData = null;

    try {
      errorData = await res.json();
      message =
        errorData.error ||
        (errorData.errors && Object.values(errorData.errors).join(", ")) ||
        message;
    } catch {
      // Non-JSON response
    }

    // Supplementary Problem 2: Token expiry handling on the frontend
    // Redirect to login if a protected request returns 401
    if (res.status === 401) {
      removeToken();
      window.dispatchEvent(
        new CustomEvent("auth:expired", {
          detail: { message: message || "Session expired. Please log in again." }
        })
      );
    }
```

```
    const err = new Error(message);
    err.status = res.status;
    err.data = errorData;
    throw err;
  }
  return res.json();
}

/**
 * Helper to build auth headers
 */
function authHeaders() {
  const token = getToken();
  return {
    "Content-Type": "application/json",
    ...(token ? { Authorization: `Bearer ${token}` } : {})
  };
}

/* ————————————————————————————————————
 Auth API Calls
———————————————————————————————————— */

export const registerUser = (userData) =>
  fetch(`${BASE_URL}/auth/register`, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
```

```
    body: JSON.stringify(userData)
  }).then(handleResponse);

export const loginUser = (credentials) =>
  fetch(`${BASE_URL}/auth/login`, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(credentials)
  }).then(handleResponse);

// Supplementary Problem 1: /me endpoint
export const getCurrentUser = () =>
  fetch(`${BASE_URL}/auth/me`, {
    method: "GET",
    headers: authHeaders()
  }).then(handleResponse);

/* ————————————————————————————————————
 Protected Task API Calls (Require Bearer Token)
———————————————————————————————————— */

export const getTasks = () =>
  fetch(`${BASE_URL}/tasks`, {
    method: "GET",
    headers: authHeaders()
  }).then(handleResponse);

export const createTask = (task) =>
```

```
  fetch(`${BASE_URL}/tasks`, {
    method: "POST",
    headers: authHeaders(),
    body: JSON.stringify(task)
  }).then(handleResponse);

export const updateTask = (id, task) =>
  fetch(`${BASE_URL}/tasks/${id}`, {
    method: "PUT",
    headers: authHeaders(),
    body: JSON.stringify(task)
  }).then(handleResponse);

export const deleteTask = (id) =>
  fetch(`${BASE_URL}/tasks/${id}`, {
    method: "DELETE",
    headers: authHeaders()
  }).then(handleResponse);
```

frontend/src/context/AuthContext.jsx

```
import { createContext, useContext, useState, useEffect } from 'react';
import {
  getToken,
  setToken,
  removeToken,
  loginUser,
  registerUser,
```

```
  getCurrentUser
} from '../api';

const AuthContext = createContext(null);

export const AuthProvider = ({ children }) => {
  const [token, setAuthTokenState] = useState(getToken());
  const [user, setUser] = useState(() => {
    const saved = localStorage.getItem('auth_user');
    return saved ? JSON.parse(saved) : null;
  });
  const [loading, setLoading] = useState(true);
  const [authNotification, setAuthNotification] = useState(null);

  // Initialize and verify user on mount if token exists
  useEffect(() => {
    const initAuth = async () => {
      const storedToken = getToken();
      if (storedToken) {
        try {
          const data = await getCurrentUser();
          setUser(data.user);
          localStorage.setItem('auth_user', JSON.stringify(data.user));
        } catch (err) {
          console.warn('Session verification failed:', err.message);
          handleLogout();
        }
      }
```

```
    setLoading(false);
  };

  initAuth();

  // Supplementary Problem 2: Listen for token expiry event from api.js
  const onAuthExpired = (event) => {
    handleLogout();
    setAuthNotification(event.detail?.message || 'Session expired. Please log in
again.');
  };

  window.addEventListener('auth:expired', onAuthExpired);
  return () => window.removeEventListener('auth:expired', onAuthExpired);
 }, []);

 const handleLogin = async (email, password) => {
  const res = await loginUser({ email, password });
  setToken(res.token);
  setAuthTokenState(res.token);
  setUser(res.user);
  localStorage.setItem('auth_user', JSON.stringify(res.user));
  setAuthNotification(null);
  return res;
 };

 const handleRegister = async (name, email, password) => {
  const res = await registerUser({ name, email, password });
```

```
    setToken(res.token);
    setAuthTokenState(res.token);
    setUser(res.user);
    localStorage.setItem('auth_user', JSON.stringify(res.user));
    setAuthNotification(null);
    return res;
  };

  // Supplementary Problem 3: Add a logout mechanism that clears the token from the client
  const handleLogout = () => {
    removeToken();
    setAuthTokenState(null);
    setUser(null);
  };

  const clearAuthNotification = () => setAuthNotification(null);

  return (
    <AuthContext.Provider
      value={{
        token,
        user,
        isAuthenticated: !!token,
        loading,
        authNotification,
        clearAuthNotification,
        login: handleLogin,
```

```
      register: handleRegister,
      logout: handleLogout
    }}
  >
    {children}
  </AuthContext.Provider>
 );
};

export const useAuth = () => {
 const context = useContext(AuthContext);
 if (!context) {
  throw new Error('useAuth must be used within an AuthProvider');
 }
 return context;
};
```

frontend/src/components/NavBar.jsx

```
import { NavLink, useNavigate } from 'react-router-dom';
import { useAuth } from '../context/AuthContext';

export default function NavBar({ darkMode, setDarkMode }) {
 const { isAuthenticated, user, logout } = useAuth();
 const navigate = useNavigate();

 const handleLogoutClick = () => {
  logout();
  navigate('/login');
```

```
};

return (
  <nav className="navbar">
    <div className="nav-brand">
      <NavLink to="/">
        <span className="brand-badge">P7</span>
        <span className="brand-title">DevTask Manager</span>
      </NavLink>
    </div>

    <div className="nav-links">
      <NavLink to="/" className={({ isActive }) => (isActive ? 'active' : '')} end>
        Home
      </NavLink>
      <NavLink to="/tasks" className={({ isActive }) => (isActive ? 'active' : '')}>
        Tasks {isAuthenticated && <span className="nav-badge">🔒</span>}
      </NavLink>
      <NavLink to="/projects" className={({ isActive }) => (isActive ? 'active' : '')}>
        Projects
      </NavLink>
      <NavLink to="/contact" className={({ isActive }) => (isActive ? 'active' : '')}>
        Contact
      </NavLink>
    </div>

    <div className="nav-actions">
      <button
```

```
  className="theme-toggle"
  onClick={() => setDarkMode(!darkMode)}
  title="Toggle Theme"
 >
  {darkMode ? '🌞' : '🌙'}
 </button>

 {isAuthenticated ? (
  <div className="user-profile-menu">
   <span className="user-pill" title={user?.email}>
    👤 {user?.name || user?.email || 'User'}
   </span>
   <button
    className="btn btn-secondary logout-btn"
    onClick={handleLogoutClick}
    title="Logout from session"
   >
    Logout
   </button>
  </div>
 ) : (
  <div className="auth-nav-buttons">
   <NavLink to="/login" className="btn btn-secondary nav-auth-btn">
    Login
   </NavLink>
   <NavLink to="/register" className="btn btn-primary nav-auth-btn">
    Register
   </NavLink>
```

```
        </div>
      )}
    </div>
  </nav>
 );
}
```

Supplementary Problem:

• Problem 1: Add a /me endpoint that returns the currently logged-in user's details using the decoded JWT.

Implementation in backend/routes/auth.js:

```
router.get('/me', authMiddleware, async (req, res, next) => {
 try {
  const user = await User.findById(req.user.id).select('-password');
  if (!user) return res.status(404).json({ error: 'User not found' });
  res.status(200).json({ user });
 } catch (err) {
  next(err);
 }
});
```

Explanation: The endpoint is protected by authMiddleware. Upon verifying the Bearer token, req.user.id is decoded. The User document is retrieved from MongoDB while omitting the password hash via .select('-password').

• Problem 2: Implement token expiry handling on the frontend — redirect to login if a protected request returns 401.

Implementation in frontend/src/api.js & frontend/src/context/AuthContext.jsx:

```
if (res.status === 401) {
 removeToken();
 window.dispatchEvent(
```

```
    new CustomEvent("auth:expired", {
      detail: { message: "Session expired. Please log in again." }
    })
  );
}
```

Explanation: Centralized handleResponse intercepts 401 responses. When an expired token or invalid signature is rejected by Express, localStorage is purged, an 'auth:expired' event fires, and AuthContext seamlessly prompts the user to re-login.

Supplementary Problem 3: Client-Side Logout Mechanism

- Problem 3: Add a logout mechanism that clears the token from the client.

Implementation in frontend/src/components/NavBar.jsx & AuthContext.jsx:

```
const handleLogout = () => {
  removeToken();
  setAuthTokenState(null);
  setUser(null);
};
```

Explanation: Clicking the 'Logout' button clears 'auth_token' and 'auth_user' from localStorage, clears context state, which triggers React re-rendering and redirects the client away from protected routes.

Github Repo: https://github.com/Bhumi3009/D25DIT090_ITUE301_AWDF